A Genetic Algorithm for Multi-Objective Optimization in Complex Course Timetabling

Ngo Tung Son*
Department of Computer and Information Sciences, Universiti Teknologi PETRONAS, Tronoh 32610, Malaysia; Information and Communication Technology Department, FPT University, Hanoi 100000, Vietnam, Email: sonnt69@fe.edu.vn

Jafreezal B Jaafar
Department of Computer and Information Sciences, Universiti Teknologi PETRONAS, Tronoh 32610, Malaysia, Email: jafreez@utp.edu.my

Nguyen Hoang Giang
Information and Communication Technology Department, FPT University, Hanoi 100000, Vietnam, Email: Giangnhhe150094@fe.edu.vn

Bui Ngoc Anh
Information and Communication Technology Department, FPT University, Hanoi 100000, Vietnam, Email: anhbn5@fe.edu.vn

ABSTRACT

The article discusses a multi-objective optimization approach for constructing an enrollment-based course timetable in universities from various sources. The model used the combination of mixed-integer and binary variables. It reaches the students' study time preferences, in which the number of the students in the same classroom is suitable for the training costs while preserving the business schedules. A compromise programming is used to transform a list of multiple objectives into a single objective function. We developed a Genetic Algorithm scheme to solve our proposed model. Finally, our proposed scheduler is evaluated with a real dataset of 3000 students. The results show the proposed method's effectiveness in both terms of solution quality and computational cost.

CCS CONCEPTS

• **Mathematics of computing** → Mathematical analysis; Mathematical optimization; Discrete optimization; Optimization with randomized search heuristics; Evolutionary algorithms.

KEYWORDS

Enrollment Based Course Timetable, Scheduling, Multi-Objective Optimization, Genetic Algorithm, Compromise Programming, Combinatorial Optimization

ACM Reference Format:
Ngo Tung Son*, Jafreezal B Jaafar, Nguyen Hoang Giang, and Bui Ngoc Anh. 2021. A Genetic Algorithm for Multi-Objective Optimization in Complex Course Timetabling. In *2021 10th International Conference on Software and Computer Applications (ICSCA 2021), February 23–26, 2021, Kuala Lumpur, Malaysia.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3457784.3457821

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICSCA 2021, February 23–26, 2021, Kuala Lumpur, Malaysia
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8882-5/21/02…$15.00
https://doi.org/10.1145/3457784.3457821

1 INTRODUCTION

Scheduling is critical in every organization. It enables the company to operate at a low cost while remaining productive. There are no exceptions when it comes to educational institutions. In educational institutions, scheduling is primarily concerned with planning teaching. The scheduler allocates resources to meet predetermined requirements such as cost or timeliness. Lecturers, teachers, classes, subjects, and curriculum are all included. Examination timetabling, and Course timetabling are commonly included in the educational timetabling dilemma [1]:

Examination timetabling: The time allocated for examinations in a given academic institution is one of the essential administrative activities in that school. It is an attempt to assign students to exams with limited resources. Qu et al. conducted an overview of the trend and development of techniques of the problem [2].

Course timetabling: Create a course schedule that has several specific time slots and assign them to every student. Babaei et al introduced a survey on the course timetabling problem in 2015 [3]. Ngo et al proposed a multi-objective optimization for solving the teaching task assignment as part of the completed scheduler [4].

In this study, we focus on the course timetabling. A non-trivial timetabling problems are normally NP-Hard problems [5]. The issue of globalization is often coupled with not being able to reach one optimal global solution. With the increase in the size of resources, the complexity of the problem also increasing, which makes it unfavorable for a systematic approach.

The authors conducted this study at FPT University in Hanoi, Vietnam, to create a scheduler that generates student schedules automatically before each semester. A student cannot learn new subjects without first completing his or her prerequisites. A class is a group of students who want to study the same subject at the same time. If the number of enrolled students is within acceptable levels, a course is considered for opening (not only a few students). For each class, the number of students ranges from 15 to 30. In reality, the number of registrations for a specific subject may not be sufficient, and students may be forced to wait for a newly opened course. As a result, students have a priority order in class placement. Students who repeat a subject are typically given a lower priority than students taking it for the first time. Students and faculty are the two most important groups. It's not easy to meet the goals of

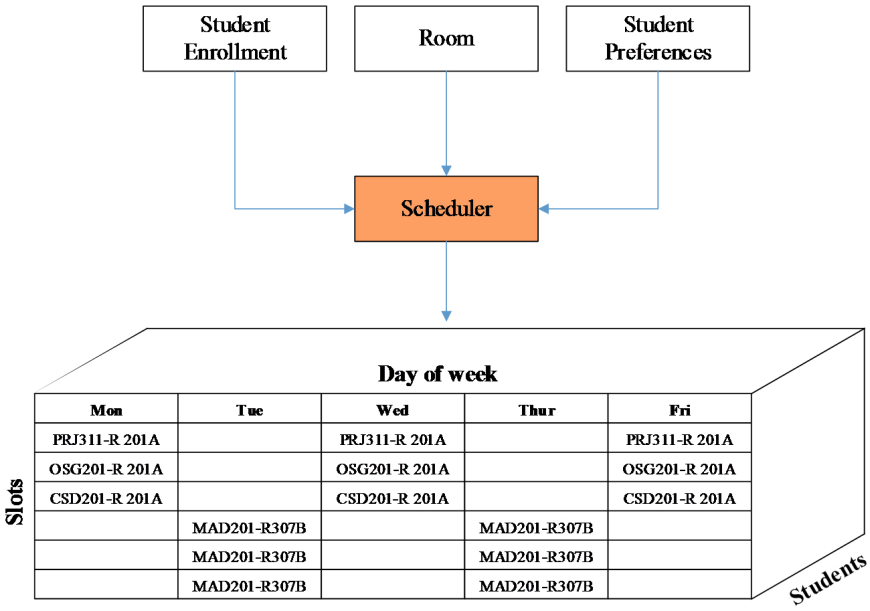

Figure 1: The proposed scheduler to timetabling.

any of the organizations mentioned above. The university still gives the student's schedule precedence over the teaching schedule of the professor.

Furthermore, since the number of lecturers is proportional to the number of classes generated, lecturers' assignment is beyond this research scope. Students may also apply their preferences for unique timeslots to be allocated to suitable classes, in addition to their enrollment details. As a result, the scheduler's input includes student lists and time preferences, enrollment details, syllabus, rooms, and conditions. The scheduler must consider several objectives to personalize the student's schedule. The input and output schedule is depicted in 1. For each day of the week, each student will obtain his or her schedule. Each time slot corresponds to the class and classroom in which students are enrolled.

1.1 Related Works

Our investigations on the current research centered on three main questions: (a) How is the timetabling problem formulated? (b) How is the specialization in models? And (c) How can we solve the educational timetabling problem?

To formulate the problems, several studies create an Integer Programming model. Andrade et al. have built a Non-Linear Binary Integer Programming mathematical model to develop the school timetabling problem, which is used to assign teaching tasks to teachers at a defined time frame [6]. Gozali et al. defined five hard and seven soft constraints for the proposed model. Where they used several binary decision variables to express the links between the pairs of resources (timeslot, lectures) and courses [7]. Phillips et al. use a general integer programming-based approach for university course scheduling, where the minimum perturbation objective is equal to the solution robustness objective [8].

There are various considerations that need to be taken into consideration when developing the optimization model for the timetabling problem. Despite the changes in the literature, the basic constraints are the similar. The most common guidelines for university class scheduling instructed are to do the following:

No violation constraints: one course to one room at a time. – the room setup meets the requirement of its assigned course. – no over room capacity- and some other business conditions.

Soft constraints: - preferences.

For example, Kristiansen et al. summarized several typical hard and soft conditions to the university timetabling problem [9]. The issues reported in the studies on modeling the timetabling problem described above are shaped by multi-Objectives, Constraints, and steps to accomplish the tasks. Akkan et al. built a university timetable with two objectives: tuition and sufficient courses [10]. They identify several soft constraints related to Room occupancy, Room capacity, minimum workdays, curriculum compactness, room stability, equity deficiencies, and average age. Mühlenthaler and Wanka presented the fairness objective and penalty objective, where fairness is described as the fair distribution of the penalty stated to the timetables among different curriculums [11].

There are many studies suggesting algorithms for optimization problems in scheduling. However, the enrollment-based course timetable problem is an NP-hard combinatorial optimization problem [5]. The metaheuristic-based techniques (such as evolutionary algorithms, tabu-search, etc.) are more suited to solve these kinds of issues than an exact or heuristic algorithm. Pillay reviews several hyper-heuristics techniques for timetabling problems such as tabu search, simulated annealing, genetic algorithms [12]. Kenekayoro designed a Greedy Ants Colony Optimization for the curriculum-based university course timetabling problem. They tested their proposed algorithm using the ITC 2007 dataset [13]. Modupe et

al. indicated that the Genetic Algorithm is a suitable method for solving timetable scheduling [14].

The reviewed researches make a common point that they are usually focused on optimizing facilities' resources instead of paying attention to individual learners' expectations. The new modern education, where the learner is centered, and all other resources/events revolve around them. We build a multi-objective optimization model, targeting not only to optimize the benefit of the operating organization but also to the students' preferences. We use integer variables to denote the link between the course-time-subject and between the student and the courses. Our proposed model is geared toward individual learners. The proposed model is a new variant of the timetabling problem. It is not suitable for reusing the algorithm proposed by previous studies. We also develop a new scheme of the Genetic algorithm to solve the proposed optimization problem.

1.2 Contribution of the Research

In this study, we build a multi-objective optimized mathematical model for the new variant of the university timetabling problem. Our model aims to maximize benefits for the organization and satisfies the maximum needs of students. We use the compromise programming approach for the maximum goal optimization problem. Researchers, as well as engineers working in operational research, can benefit from this research. The model is described in the second section, together with the approach to the multi-objective optimization problem. Section 3 of the paper describes a scheme of the Genetic algorithm for solving the proposed model. We test the proposed model and algorithm by scheduling FPT University students in the spring semester of 2020. The test results are shown in section 4 of the paper. The rests are discussions and conclusions.

2 PROBLEM FORMULATION

2.1 MOP for Timetabling Problem

Some of the variables used in the model are defined below:

S is the number of subjects.

M is the number of students.

T denotes the number of time-slots

R is the number of class rooms.

F is the number of conditions of the subjects.

$\mathcal{H}$ and g respectively denote the minimum and maximum number of students in the same course.

Decision variables $C = \{C_s | C_s \geq 0,\ C_s \in \mathbb{N},\ s = 1 \ldots S\}$ denotes the number of courses in particular subject s^{th}.

$L = \{l_{s,f} | l_{s,f} \in \{0,1\},\ s = 1 \ldots S, f = 1 \ldots F\}$ denotes the required conditions for subjects. Where: $l_{s,f} = 1$ means the subject s^{th} requires condition f^{th}. 0 otherwise.

$K = \{k_{m,t} | k_{m,t} = 0 \ldots 5,\ m = 1 \ldots M, t = 1 \ldots T\}$ represents the preferences of the students on the corresponding time-slots. Where $k_{m,t} = 0$ the student m^{th} cannot study at time-slot t^{th}. $k_{m,t} > 0$ denotes the preferences of the student m^{th} to time-slot t^{th}.

$U = \{u_{r,f} | u_{s,f} \in \{0,1\},\ r = 1 \ldots R, f = 1 \ldots F\}$ denotes the conditions for corresponding class-rooms. Where: $u_{s,f} = 1$ means the room r^{th} satisfy the condition f^{th}. 0 otherwise.

$A = \{a_{m,s} |\ a_{m,s} = 0 \ldots 5,\ m = 1 \ldots M,\ s = 1 \ldots S\}$ stands for the registrations of students for particular course. Where: $a_{m,s} = 0$ if the student m^{th} is not going to study subject s^{th}. $a_{m,s} > 0$ denotes the priority degree of the student m^{th} to be arranged to a course of subject s^{th}.

Decision Variables $X = \{x_{m,s,c} | x_{m,s,c} \in \{0,1\},\ m = 1 \ldots M, s = 1 \ldots S, c = 1 \ldots C_s\}$ Where $x_{m,s,c} = 1$ if the student m^{th} is assigned to the course c^{th} for the subject s^{th}. $x_{m,s,c} = 0$ otherwise.

Decision Variables $Y = \{y_{c,s,t,r} | y_{c,s,t,r} \in \{0,1\},\ t = 1 \ldots T, r = 1 \ldots R, s = 1 \ldots S, c = 1 \ldots C_s\}$ Where $y_{c,s,t,r} = 1$ if the course c^{th} of the subject s^{th} assigned to room r^{th} at the time-slot t^{th}. $y_{c,s,t,r} = 0$ otherwise.

$X_m = \{x_{m,s,c} \in X | x_{m,s,c} \in \{0,1\},\ s = 1 \ldots S,\ c = 1 \ldots C_s\}$ is the schedule of the student m^{th}.

$Y_m = \{y_{c,s,t,r} \in (Y \infty X_m)\}$ is the schedule of the courses which are participated in by the student m^{th}. ∞ denotes the natural join operation.

The objective functions are respectively defined as:

The more students placed in the classroom, the better. Students with higher priority are more likely to be placed in the class.

$$max\left(\sum_{m=1}^{M}\sum_{s=1}^{S}\sum_{c=1}^{C_s}\left(x_{m,s,c} * a_{m,s}\right)\right)$$

Maximize the preferences of the students to the time-slots.

$$max\left(\sum_{s=1}^{S}\sum_{c=1}^{C_s}\sum_{t=1}^{T}\sum_{r=1}^{R}\sum_{m=1}^{M}\left(x_{m,s,c} * k_{m,t} * y_{c,s,t,r}\right)\right)$$

The number of students in the courses of the same subject is not too different.

$$min\left(\left(\left|\sum_{m=1}^{M} x_{m,s,c} - \frac{\sum_{m=1}^{M}\sum_{c=1}^{C_s} x_{m,s,c}}{C_s}\right|\right)\right)\ \forall s = 1 \ldots S,\ c = 1, \ldots C_s$$

The number of parts of the days, student have to go to school are minimized.

$$max\left(\sum_{m=1}^{M} pod\left(X_m, Y_m\right)\right)$$

Where: $pod(X_m, Y_m)$ is the function that returns the score of the student m^{th} 's schedule based on the parts of day the student has to go to the school.

Minimize the numbers of created courses.

$$\min\left(C_s\right) \qquad \forall s = 1 \ldots S$$

Subjects to:

2 different courses cannot be arranged at the same room, at the same time-slot.

$$\sum_{s=1}^{S}\sum_{c}^{C_s} y_{c,s,t,r} \leq 1 \qquad \forall t = 1 \ldots T,\ r = 1 \ldots R$$

The number of enrolled students in the same course must be restricted.

$$\mathcal{H} \leq \sum_{m=1}^{M} x_{m,s,c} \leq \mathcal{G} \quad \forall s = 1 \ldots S, c = 1 \ldots C_s$$

Students are not arranged to study courses they do not register for.

$$\sum_{s}^{S}\sum_{c}^{C_s} x_{m,s,c} \le \min\left(a_{m,s}, 1\right) \qquad \forall\ m = 1 \ldots M$$

Students are not assigned to learn at the time-slot they have no preference.

$$\sum_{s=1}^{S}\sum_{c=1}^{C_s}\sum_{r=1}^{R}\left(x_{m,s,c} * y_{c,s,t,r} * k_{m,t}\right) \le 1 \ \ \forall\ m = 1 \ldots M,\ t = 1 \ldots T$$

Course only assigned to the room that satisfy the required conditions.

$$y_{c,s,t,r} * u_{r,f} \ge\ l_{s,f} \quad \forall\ s = 1 \ldots S, c = 1 \ldots\ C_s,\ t = 1 \ldots T, r = 1 \ldots R,\ j$$

2.2 Compromise Programming for Proposed MOP

The proposed model is multi-objective programming (MOP). It is difficult to define a solution that is the minimum of all objective functions. There are two main approaches to solving the MOP problem: preference method and non-preference method, as Hwang mentioned [15]. The first approach is more suitable for most three objective functions because it is easy to visualize the decision space solutions. The second is the preference that works based on an assumption of no existing decision-maker but available a naturally compromised solution to all objectives. It fits this situation where the model contains many objectives.

In this part of the paper, we describe how we apply compromise programming to transform the MOP into a single object problem. The Compromise programming approach is based on finding the minimum distance between the solution O to an introduced idea point E [16]. Ngo et al. also applied a similar mechanism in their team selection problem [17]. In this research, for object the balance of students, we consider the students difference in each exclusive subject as different object. So for S subject, we have S objective for objective 3, and the dimensions of ideal point and solution become 4+S. The ideal point denoted as $E = \{e_i | i = 1 \ldots (4+S)\}$, and the solution $O = \{o_i | i = 1 \ldots (4+S)\}$

where:

$$e_i = \begin{cases} \sum\limits_{m=1}^{M}\sum\limits_{s=1}^{S} a_{m,s}\ if\ i = 1 \\ \sum\limits_{m=1}^{M}\sum\limits_{t=1}^{T} k_{m,t}\ if\ i = 2 \\ 0\ if\ i = 3 \ldots (2+S) \\ M^{*}\square\ ij\ i = (3+S) \\ 0 \qquad otherwise \end{cases}$$

$$o_i = \begin{cases} \sum\limits_{m=1}^{M}\sum\limits_{s=1}^{S}\sum\limits_{c=1}^{C_s}\left(x_{m,s,c} * a_{m,s}\right)\ \ if\ \ i = 1 \\ \sum\limits_{s=1}^{S}\sum\limits_{c=1}^{C_s}\sum\limits_{t=1}^{T}\sum\limits_{r=1}^{R}\sum\limits_{m=1}^{M}\left(x_{m,s,c} * k_{m,t} * y_{c,s,t,r}\right)\ \ if\ \ i = 2 \\ \sum\limits_{c}^{C_{(i-2)}}\left|\sum\limits_{m=1}^{M} x_{m,(i-2),c} - \frac{\sum_{m=1}^{M}\sum_{c=1}^{C_{(i-2)}} x_{m,(i-2),c}}{C_{(i-2)}}\right|\ \ if\ \ i = 3 \ldots (2+S) \\ \sum\limits_{m=1}^{M} pod\left(X_m, Y_m\right)\ \ if\ \ i = (3+S) \\ \sum\limits_{s=1}^{S} C_s - \sum\limits_{s=1}^{S}\sum\limits_{m=1}^{M}\frac{\max\left(\min\left(a_{m,s}, 1\right), 0\right)}{g}\ \ \ if\ \ \ i = (4+\mathrm{S}) \end{cases}$$

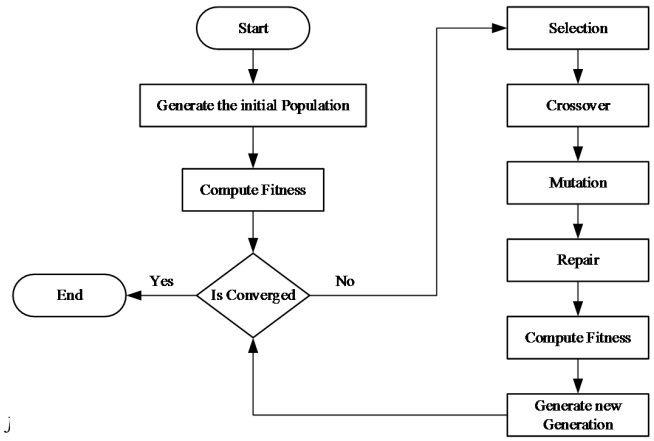

Figure 2: The flow of the proposed generic algorithm.

Objective function becomes:

$$\min\left(\sqrt{\sum_{i=1}^{(4+\mathrm{S})} w_i * norm_i (e_i - o_i)^2}\right)$$

Where $norm_i$ represents a normalization function that rearranges the values of the dimension i^{th} in the distance function to a particular range. w_i stands for the weighted parameters of the dimension i^{th} in the distance function.

3 PROPOSED ALGORITHM

Evolutionary algorithms (EA) are mainstream stochastic methods for MOP [18]. In EA's family, Genetic Algorithms (GA) are suitable for many engineering applications [19]. The genetic algorithm is a population-based heuristic method extensively used in scheduling problems. It searches a solution space for the optimal solution to a problem. The algorithm's idea is to find the best individuals that fit the problem's goal through the hybridization of the best genes from the parent. The flow of the proposed algorithm illustrated in Figure 2

3.1 Genetic Representation and Fitness Function

Chromosome is represented as a compound of three small matrixes:

A represents a matrix of R rows and T columns, rows r^{th} represents the courses that using room r^{th}. $Cell\ (r,\ t) > 0$ if there is a course that using room r^{th} at slot t^{th}, or $Cell\ (r,\ t) = 0$ if room r^{th} is not being used in slot t^{th}.

V represents a vector of $\sum\limits_{s=1}^{S}\sum\limits_{m=1}^{M}(\max(\min(a_{m,s}, 1), 0))/\mathcal{H}$ dimensions, where the whole vector represents all the possible courses can be created. And dimension c^{th} represents whether course c^{th} being used or not, 1 if course c^{th} is used, 0 otherwise.

B represents matrix of M rows and S columns, the row m^{th} denotes the course assigned to the student m^{th}. $Cell\ (m,\ s) > 0$ if there is a course of subject s^{th} that student m^{th} is assigned to, or 0 if student m^{th} is not enrolled to subject s^{th}.

Denote $MObj = \{MObj_1, MObj_2 \ldots, MObj_{S+4}\}$ Where :

$$MObj_i = \begin{cases} \sum_{m=1}^{M}\sum_{s=1}^{S} a_{m,s} & if\ i = 1 \\ \sum_{s=1}^{S}\sum_{c=1}^{C_s} x_{m,s,c} & \\ \sum_{i=1} \max_{t=1\ldots T} k_{m,t} & if\ i = 2 \\ \max(|\max - \frac{\sum_{m=1}^{M}\sum_{c=1}^{C_s} x_{m,s,c}}{C_s}|, |\min - \frac{\sum_{m=1}^{M}\sum_{c=1}^{C_s} x_{m,s,c}}{C_s}|) & if\ i = 3\ldots(2+S) \\ \sum_{s=1}^{S} \frac{\sum_{m=1}^{M}(\max(\min(a_{m,s},1),0))}{\mathcal{H}} - \sum_{s=1}^{S}\sum_{m=1}^{M} \frac{\max(\min(a_{m,s},1),0)}{g} & if\ i = +S \end{cases}$$

Where:

$maxn(k_{m,t}, \sum_{s=1}^{S}\sum_{c=1}^{C_s} x_{m,s,c})$ returns the $\sum_{s=1}^{S}\sum_{c=1}^{C_s} x_{m,s,c}$ biggest value in $k_{m,t}$.

$\sum_{c}^{C_s} \max(|\mathcal{H} - \frac{\sum_{m=1}^{M}\sum_{c=1}^{C_s} x_{m,s,c}}{C_s}|, |g - \frac{\sum_{m=1}^{M}\sum_{c=1}^{C_s} x_{m,s,c}}{C_s}|)$ returns the bigger value between 2 value.

$\frac{\sum_{m=1}^{M}\sum_{c=1}^{C_s} x_{m,s,c}}{C_s}$ is the average number of students in subject s^{th}, where s is the subject of the current course. The fitness values $P_i^{(g)}.fitness$ could be bias due to the difference in ranges of its dimensions. We normalize features to the range [0,1]. The Fitness function of the individual can be defined as:

$$P_i^{(g)}.fitness = \sqrt{\sum_{i=1}^{(4+S)} w_i * \left(\frac{e_i - o_i}{MObj_i}\right)^2}$$

Where $norm_i(e_i - o_i) = \frac{e_i - o_i}{MObj_i}\ \forall\ i = 1\ldots 4+S$.

3.2 Genetic Operation

There are two groups of decision variables introduced in this problem. The group of integers variables represents the number of classes open for each subject (inset C). The second group is binary variables representing the distribution of objects such as students, timeslots, and classes (in the sets X and Y)—the first group assigned the value before the second group in the genetic operations.

Denote:

U represents the size of population

$P^e = \{p_i^e |\ i = 1..U\}$ as the population at generation e^{th}

p_i^e as the individual ith of the population at generation e^{th}

φ as the tournament size for selection.

k as the mutation rate.

∂_f is the set of courses that course c^{th} that teach subject have condition f^{th}.

γ_s is the set of courses that teach subject s^{th}.

α_c is the set of students that is assigned to course c^{th}.

$Q_{m,t} = \{Q_{m,t}|^{Q_{m,t} = 0\ldots 1,\ t = 1\ldots T,\ m = 1\ldots M}\}$ Where: $Q_{m,t}^{(i)} = \begin{cases} 0\ if\ student\ m\ busy\ in\ slot\ t \\ 1 \quad otherwise \end{cases}$

There are 7 steps of the algorithm as following:

Step 1- Generate the initial population:

- For every row r^{th} in A that represent room have condition f^{th} contains $\partial_f\ and\ G - |\partial_k|$ number 0, so fill randomly $\partial_f\ and\ G - |\partial_k|$ number 0 to those rows for each condition f^{th}.
- For each subject s^{th} randomly chose a n (randomly) number of course from γ_s, which n is bigger than $\sum_{m=1}^{M}(\max(\min(a_{m,s},1),0))/g$ and smaller than $\sum_{m=1}^{M}(\max(\min(a_{m,s},1),0))/\mathcal{H}$ And put value 1 in V for each course is chosen.
- In matrix B for each cell (m, s) where student m^{th} when to enroll to subject s^{th} chose a random course which is chosen in V (priority the course that have the smallest number of students, for course that have number of student$\rangle g$ then do not chose it), and that course is in a slot that student is not busy. If cannot find one, then set the value of the cell to 0.
- After filling both matrixes, apply repair operator to ensure the constraint.

Step 2 - Selection: Randomly select φ individuals from P^e and return the best individual base on fitness value among them.

Step 3 - Crossover : Denote p_{father}^e and p_{mother}^e are parents to crossover:

- The set of numbers in every row r^{th} with the condition f^{th} of A is pre-mutation, so to crossover, we have to ensure its pre-mutation, mean if we replace cell (r, t) with other value, we have to replace the value that contains the new value with the old value. After switch room-timeslot of a course c^{th}, it is needed to change the Q_m for each student m^{th} in α_c according to the new slot of course c^{th}. However, there are some students that is already at the new slot, so they need to move to another course teach the same subject s^{th} that fill the condition for student m^{th} to enroll (do not have more student than max, student m^{th} not busy at the slot of this course, being used), for those can not find the alternative course, assigned 0 to the according cell in B.
- For vector V, the crossover method is to swap all the cell of each course of subject s^{th} (if chose to crossover) to each other. For those course that is being disable despite having students assigned in. Move all of the students out of this course (to another course or unenrolled them).
- For matrix B, because the set of number in each column s^{th} is γ_s and is fix, so the crossover method does not pose a problem. However, when moving a student out cause the number of students in course c^{th} , it is needed to find another student to enroll in this course to replace this student, if cannot find one then skip this crossover.

Step 4 - Mutation: For each individual p_i^e, have rate k to swap some element in matrix A and B in exchange to a random number that in the set of numbers that correspond to this cell. As for vector V, it flips some element in vector (from 1 to 0, 0 to 1). After performing crossover and mutation must be applied repair operator to ensure there are no invalid results during the processing.

Step 5 - Repair Process: The Process consists two steps:

- For every course they have the number of student smaller than min, it has to move all the student enroll in the course to another course that pass the condition to let that student

Table 1: The details of 10-time slots defined at the FPT University for a week

DoWTime-Slot	Monday	Tuesday	Wednesday	Thursday	Friday	Part of the day
1	M1	M4	M1	M4	M1	Morning
2	M2	M4	M2	M5	M2	
3	M3	M5	M3	M5	M3	
4	E1	E4	E1	E4	E1	After Noon
5	E2	E4	E2	E5	E2	
6	E3	E5	E3	E5	E3	

in. For those who cannot find a course, un-enroll them to this subject s^{th}.

- For each student m^{th} that want to enroll subject s^{th} but do not assigned to a course that teach course s^{th} yet. Then find a suitable course c^{th} that teach subject s^{th}, student m^{th} is not busy at the slot of this slot, then move student in course c^{th}. If course c^{th} already have max number of students, then its have to chose a random student enroll in it, and move that student to another course, no student can move out then find another course. If course c^{th} is empty, then fill the course with random number of student (greater min, lesser max student and priority those who are not enroll). If cannot find a suitable course then leave student m^{th} un-enroll.

Step 6 - Constraint validation checking: If there is an individual $P_i^{(g+1)}$ that violates any constraints, then it is removed from the set of results.

Step 7- Finally: Repeat 2, 3, and 4 ,5 and set $g = g + 1$ until $b^g = b^{g-1} = \ldots = b^{g-G}$.

4 EXPERIMENTS

To conduct experiments to evaluate the proposed model and algorithm, we collected the data of more than 3000 students in the IT department at FPT University, consisting of almost 150 subjects. We collected more than 12000 application forms from 3000 students enroll in different subjects from 150 subjects. We experiment on two different scales. The first uses the data of all the students in the department. The second only uses data of students who in the same semester, which is semester 3, consists of more than 500 students and only five subjects. These classes are assigned to 10 timeslots described in Table I.

The experience is implemented in python3 and executed on the computer with detailed configuration as CPU Intel core i5-8300H @2.30Hz, 8GB. We evaluated both of the proposed algorithms on the tested dataset. The pod function implemented as following:

$$pod\,(X_m, Y_m) = \begin{cases} \beth\ If\ ((1 \le n \le 3)\ and\ \ (r = 1))\ or\ ((4 \le n \le 6)\ and\ \ (r = 2)) \\ \qquad or\ ((9 \le n \le 10)) \\ or\ ((7 \le n \le 8)\ and\ (r = 3))\ or\ ((7 \le n \le 8)\ and\ (r = 3)) \\ \frac{\beth}{5}\ \text{If}\ ((1 \le n \le 3)\ and\ (r = 2))\ or\ ((4 \le n \le 6)\ and\ (r = 3)) \\ \qquad \frac{\beth}{2}\ \text{If}\ ((7 \le n \le 8)\ and\ (r = 4)) \\ 0\ \text{If}\ ((\beth \le n \le 3)\ and\ (r \ge 3))\ or\ \ ((4 \le n \le 6)\ and\ \ (r = 4)) \end{cases}$$

$$\forall\ m = 1 \ldots M$$

Where: $\beth = 100$, $n = \sum_s^S \sum_c^{C_s} \sum_r^R \sum_t^T (y_{c,s,t,r})$ and $r = NumPod(Y_m)$

With $NumPod(Y_m) = cp(\{M1, M2, M3\}, Y_m) + cp(\{E1, E2, E3\}, Y_m) + cp(\{M4, M5\}, Y_m) + cp(\{E4, E5\}, Y_m)$and

$$cp\,(Time, Y_m) = \min\left(\left(\sum\nolimits_{s=1}^{S} \sum_{c=1}^{C_s} \sum\nolimits_{r=1}^{R} \sum_{t \in Time} y_{c,s,t,r}\right), 1\right)$$

Many parameters govern the GA. The number of students who can enroll needs more priority, which requires a series of weights for the objective. We execute the algorithms several times with the collected dataset to indicate the best set of parameters and weights shown in Table II. Part A of the table displays the parameters, while the remaining displays the weights. For the different datasets, decision-maker can customize their preferences via a different set of parameters.

Meta-heuristic algorithms do not guarantee to find the optimal solution with different initialization values. Therefore, we run the algorithm 15 times with different initializations to find the schedule for 500 students in a semester. The results are shown in Figure 3. It can be observed that the fitness value of these algorithms does not change over different executions, which offers the stability of the algorithm. The time for the algorithm to be executed fluctuate around 2 minutes in a dataset of 500 students. However, with a bigger dataset, the algorithm takes much more time to complete, like it takes about 2 hours to run with 3000 students' data.

Part A of Figure 4 illustrates the change of fitness value over generations with a dataset of 500 students. It can be seen that the algorithm reaches stability around generation 50th (which is typical of 15 executions) and archives the best solution after 700 loops. The change of fitness value over generations with the experience of 3000 students has the same phenomenon with the knowledge of 500 students (part B of Figure 4). However, unlike the smaller scale dataset, with 3000 students and 150 subjects, the number of students unable to enroll in a subject exists. Simultaneously, the previous experience can find a solution where the number of unenrolled reaches 0. According to our observations, the algorithm converges quite early. This may be a sign of the lack of diversity of genetic algorithms.

Figure 5 gives the number of unenrolled compares to all the application of subjects which have unenrolled. Simultaneously, some subjects have several unenrolled insignificant compared to all the number of students who went to enroll in this subject. Some subjects do not have any students can enroll (in Figure 6). However,

Table 2: Parameters and weights to execute the algorithms.

Parameters to control the genetic operations	
Parameter	Value
Number of population	10
Exchange genes rate	0.9
Mutation rate	0.1
Selection size	3
Number of elites	1
B) Weighted parameters to express the decision maker preferences.	
Number of enroll students w_1	100000
Student's preferment w_2	1
Student's schedule in part of the day w_3	1
Number of courses w_4	1
The different of students in courses $w_5, \ldots, w_{4+S}$	1

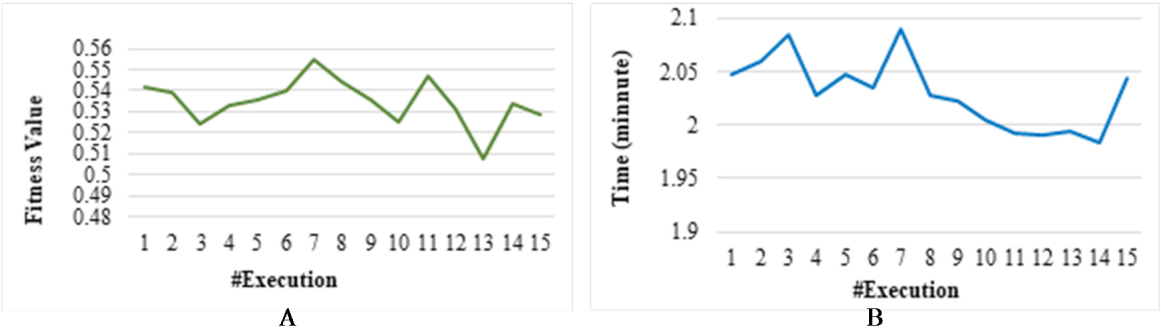

Figure 3: Outputs of executing 15 times. A- Fitness values and B- Execution time over different runs.

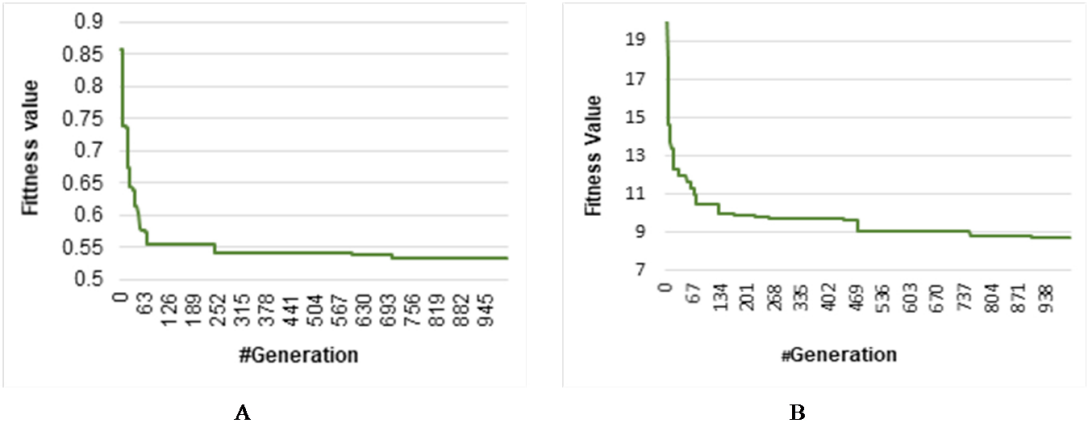

Figure 4: Fitness values changed over generations in GA. A) Dataset of 500 students. B) Dataset of 3000 students.

this case only happened for subjects with several students who want to register too small (not enough to open more than one course), and with a large number of applications, it is hard for the algorithms to schedule it.

In Figure 6, it's group courses that have the same number of students. It can be seen that courses with 28 to 30 students are the majority. However, there are still exist several courses that have enough students barely to create a course (15 students) because the objective largest number of students is the priority (more course, more slot for a student to enroll).

Figure 7 illustrates students' preferences of experience 3000 students by statistic the number of students with the same preferences.

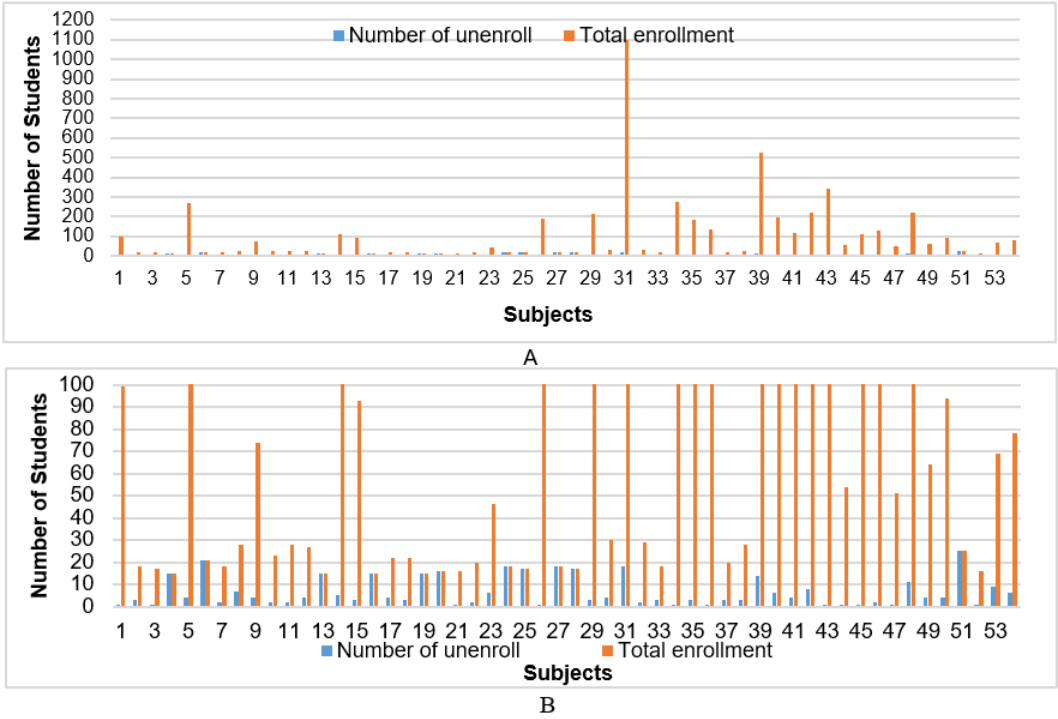

Figure 5: Number of unenrolled compare total enrolment of different subjects. A) Scale of 3000 students. B) Scale of 500 students

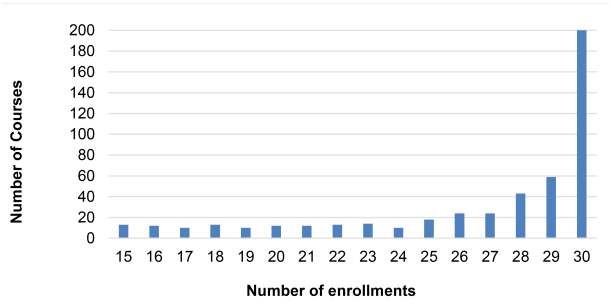

Figure 6: Counting the Number of courses and their number of enrollments

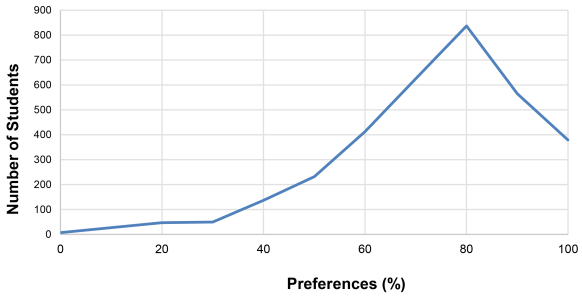

Figure 7: The preferences of students

Figure 8 shows the pod score (part of the students' day score). In Figure 7, it can be seen that the majority of students have preferences larger than 50% that satisfy the objective to maximize the preferences of the students. However, in Figure 8, the number of students who have the pod score in the lower haft (0 and 20) takes most students. To explain this, we look at the preferences of each student to the timeslot. The timeslots that rated with high preferences do not stay in the same part of the day, which makes objective maximize students' preferences and minimized the number of parts of the day's student has to go to school contradict with each other.

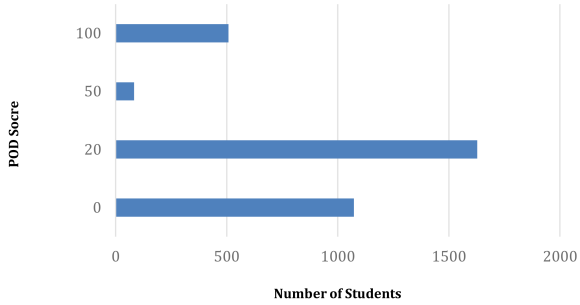

Figure 8: The number of students and corresponding pod scores

Campus: FU-HL
Student: sonnt5 View

YEAR 2021	MON	TUE	WED	THU	FRI	SAT	SUN
WEEK 22/02 To 28/02	22/02	23/02	24/02	25/02	26/02	27/02	28/02
Slot 1	SE1434 -PRJ321 at AL-R206 (Attended)	SE1422 -PRN292 at AL-L206 (Attended)	SE1434 -PRJ321 at AL-R206 (Attended)	SE1422 -PRN292 at AL-L206 (Attended)	SE1434 -PRJ321 at AL-R206 (Attended)	-	-
Slot 2	SE1517 -PRO192 at DE-215 (Attended)	SE1422 -PRN292 at AL-L206 (Attended)	SE1517 -PRO192 at DE-215 (Attended)	IS1314 -DBW301 at AL-L205 (Attended)	SE1517 -PRO192 at DE-215 (Attended)	-	-
Slot 3	SE1436 -PRJ321 at AL-R306 (Attended)	IS1314 -DBW301 at AL-L205 (Attended)	SE1436 -PRJ321 at AL-R306 (Attended)	IS1314 -DBW301 at AL-L205 (Attended)	SE1436 -PRJ321 at AL-R306 (Attended)	-	-
Slot 4	SE1421 -PRN292 at AL-L302 (Attended)	IS1309 -DBW301 at AL-L207 (Attended)	SE1421 -PRN292 at AL-L302 (Attended)	IS1309 -DBW301 at AL-L207 (Attended)	SE1421 -PRN292 at AL-L302 (Attended)	-	-
Slot 5	SE1503 -PRJ301 at AL-R307 (Attended)	IS1309 -DBW301 at AL-L207 (Attended)	SE1503 -PRJ301 at AL-R307 (Attended)	IS1312 -DBW301 at AL-L205 (Attended)	SE1503 -PRJ301 at AL-R307 (Attended)	-	-
Slot 6	IS1316 -DBW301 at AL-L203 (Attended)	IS1312 -DBW301 at AL-L205 (Attended)	IS1316 -DBW301 at AL-L203 (Attended)	IS1312 -DBW301 at AL-L205 (Attended)	IS1316 -DBW301 at AL-L203 (Attended)	-	-

Figure 9: The generated schedule for a particular student visualized on the webpage.

5 CONCLUSION

We presented a university scheduling aid in this research. The proposed model is a modern version of the enrollment-based course scheduling problem as compared to previous research. Our goal-setting roles provide access to both individual student preferences and business objectives. Since each meta-heuristic algorithm scheme is tailored to a specific model, it isn't easy to reuse previously used algorithms. We designed a new scheme for the Genetic Algorithm to solve the problem. Although, based on our observation, it still has several limitations. The least installed algorithm worked well with our tested dataset. Students' expectations for the faculty of interest are also ignored in this model due to the research schedule's division into two phases. Apart from fine-tuning our algorithm, this is something we'll be doing soon. We intend to develop a scheduling management system that can assist the decision-maker in manually adjusting the resulting solution because no metaheuristic algorithm (including genetic algorithm) guarantees a globally optimal solution to the timetabling problem. 9 shows the first version of the developing system. A webpage that helps the student to visualize the generated schedule. The completed system makes stakeholders quickly participated in the post-processing.

REFERENCES

[1] Pillay, N. (2014). A review of hyper-heuristics for educational timetabling. Annals of Operations Research, 239(1), 3–38. doi:10.1007/s10479-014-1688-1.

[2] R. Qu; E. K. Burke; B. McCollum; L. T. G. Merlot; S. Y. Lee (2009). A survey of search methodologies and automated system development for examination timetabling. , 12(1), 55–89. doi:10.1007/s10951-008-0077-5.

[3] Babaei, Hamed; Karimpour, Jaber; Hadidi, Amin (2015). A survey of approaches for university course timetabling problem. Computers & Industrial Engineering, 86(), 43–59. doi:10.1016/j.cie.2014.11.010

[4] Son, Ngo T.; Jaafar, Jafreezal; Aziz, Izzatdin A.; Anh, Bui N. 2021. "A Compromise Programming for Multi-Objective Task Assignment Problem" Computers 10, no. 2: 15. https://doi.org/10.3390/computers10020015

[5] Cooper, T.B., Kingston, J.H.: The complexity of timetable construction problems. In: Proc. of the 1st Int. Conference on the Practice and Theory of Automated Timetabling (ICPTAT '95). (1995) 511–522

[6] Andrade, P. R. L., Steiner, M. T. A., & Góes, A. R. T. (2019). Optimization in timetabling in schools using a mathematical model, local search and Iterated Local Search procedures. Gestão & Produção, 26(4), e3421. https://doi.org/10.1590/0104-530X3241-19.

[7] Gozali, Alfian A.; Kurniawan, Bobby; Weng, Wei; Fujimura, Shigeru (2019). Solving university course timetabling problem using localized island model genetic algorithm with dual dynamic migration policy. IEEJ Transactions on Electrical and Electronic Engineering, (), tee.23067–. doi:10.1002/tee.23067

[8] Phillips, A. E., Walker, C. G., Ehrgott, M., & Ryan, D. M. (2016). Integer programming for minimal perturbation problems in university course timetabling. Annals of Operations Research, 252(2), 283–304. doi:10.1007/s10479-015-2094-z.

[9] Kristiansen, S., & Stidsen, T. R. (2013). A comprehensive study of educational timetabling—A survey. Department of Management Engineering, Technical University of Denmark. (DTU Management Engineering Report; No. 8.2013)

[10] Akkan, C., & Gülcü, A. (2018). A bi-criteria hybrid Genetic Algorithm with robustness objective for the course timetabling problem. Computers & Operations Research, 90, 22–32. doi:10.1016/j.cor.2017.09.007.

[11] Mühlenthaler, M., & Wanka, R. (2014). Fairness in academic course timetabling. Annals of Operations Research, 239(1), 171–188. doi:10.1007/s10479-014-1553-2.

[12] Pillay, N. (2016). "A review of hyper-heuristics for educational timetabling." Annals of Operations Research 239(1): 3-38.

[13] Kenekayoro, P. and G. Zipamone (2016). "Greedy Ants Colony Optimization Strategy for Solving the Curriculum Based University Course Timetabling Problem." arXiv preprint arXiv:1602.04933.

[14] A. O. Modupe, O. E. Olusayo, and O. S. Olatunde, "Development of a university lecture timetable using modied genetic algorithms approach," International Journal of Advanced Research in Computer Science and Software Engineering, vol. 4, no. 9, pp. 163{168, 2014.

[15] Ching-Lai Hwang; Abu Syed Md Masud (1979). Multiple objective decision making, methods and applications: a state-of-the-art survey. Springer-Verlag. ISBN 978-0-387-09111-2.

[16] Zeleny, M. (1973), "Compromise Programming", in Cochrane, J.L.; Zeleny, M. (eds.), Multiple Criteria Decision Making, University of South Carolina Press, Columbia, pp. 262–301.

[17] Ngo, T.S.; Bui, N.A.; Tran, T.T.; Le, P.C.; Bui, D.C.; Nguyen, T.D.; Phan, L.D.; Kieu, Q.T.; Nguyen, B.S.; Tran, S.N. Some Algorithms to Solve a Bi-Objectives Problem for Team Selection. Appl. Sci. 2020, 10, 2700.

[18] Lücken, Christian; Barán, Benjamín; Brizuela, Carlos (2014). A survey on multi-objective evolutionary algorithms for many-objective problems. Computational Optimization and Applications, 58(3), 707–756. doi:10.1007/s10589-014-9644-1

[19] K. F. Man, K. S. Tang, S. Kwong, Genetic algorithms: concepts and applications, IEEE Transactions on Industrial Electronics 43 (5) (1996) 519-534.